

GUÍA PRÁCTICA

Actividad – Módulo Docentes del SGA

Objetivo

El módulo de **Alumnos** ya permite registrar, visualizar, editar y eliminar información.

Ahora vas a aplicar los mismos conceptos para construir un segundo módulo del Sistema de Gestión Académica: **Docentes**.

La actividad busca que puedas reutilizar lo aprendido y adaptarlo a una nueva entidad.

1. Punto de partida

El proyecto ya cuenta con:

```
SGA
├── index.html
├── css/
│   └── estilos.css
└── js/
    └── alumnos.js
```

Deberás incorporar:

```
js/
├── alumnos.js
└── docentes.js
```

Importante: no trabajes el módulo Docentes dentro de alumnos.js.

2. Crear el archivo docentes.js

Crea:

`js/docentes.js`

Y vincúlalo desde el HTML:

```
<script src="js/docentes.js"></script>
```

Colocalo después de `alumnos.js`.

3. Definir los datos de un docente

Un docente tendrá los siguientes datos:

ID
Nombre
Especialidad
Correo

Por ejemplo:

```
{
  id: 1,
  nombre: "Laura",
  especialidad: "Programación",
  correo: "laura@mail.com"
}
```

4. Crear el array de docentes

En `docentes.js`:

```
const docentes = [];
```

Este array representará, por ahora, nuestra colección de docentes.

5. Crear el formulario

Incorporá al HTML un formulario para registrar docentes.

Debe permitir ingresar:

- Nombre
- Especialidad
- Correo

Por ejemplo:

ALTA DE DOCENTE	
Nombre:	[_____]
Especialidad:	[_____]
Correo:	[_____]
[Registrar docente]	

Podés reutilizar la estructura visual del formulario de alumnos.

6. Obtener los datos del formulario

Al enviar el formulario, obtené los valores utilizando JavaScript.

Recordá el patrón que utilizamos con alumnos:

```
document.querySelector(...)
```

y:

```
.value
```

También utilizá:

```
.trim()
```

para evitar espacios innecesarios.

7. Crear el objeto docente

Con los datos obtenidos del formulario, construí un objeto:

```
const docente = {  
  id: Date.now(),  
  nombre: ...,  
  especialidad: ...,  
  correo: ...  
};
```

Recordá

En el módulo Alumnos utilizamos:

```
Date.now()
```

para generar un identificador mientras todavía no tenemos una base de datos.

8. Agregar el docente

Una vez creado el objeto, incorporalo al array:

```
docentes.push(docente);
```

Después deberás guardar la información en localStorage.

9. Persistencia

Los docentes deberán permanecer almacenados aunque se actualice la página.

Utilizá:

```
localStorage.setItem()
```

y:

```
JSON.stringify()
```

10. Recuperar los docentes

Creá una función:

`obtenerDocentes()`

Su función será recuperar los datos almacenados en `localStorage`.

Recordá que para convertir nuevamente el JSON en un array utilizamos:

`JSON.parse()`

Y si todavía no existen docentes almacenados, la función debería devolver:

`[]`

11. Mostrar los docentes

Creá una función:

`mostrarDocentes()`

Los docentes deberán visualizarse en una tabla.

La tabla tendrá:

ID	Nombre	Especialidad	Correo
1	Laura	Programación	laura@mail.com
2	Martín	Bases de datos	martin@mail.com

Utilizá JavaScript para generar las filas dinámicamente.

12. Mostrar los docentes al cargar la página

Recordá el problema que solucionamos con alumnos.

Si actualizamos la página:

Los docentes almacenados deben seguir apareciendo.

Por lo tanto, al cargar el archivo JavaScript deberás:

1. obtener los docentes;
2. mostrarlos.

13. Validaciones

Aplicá al formulario de docentes algunas de las validaciones que ya incorporamos al módulo Alumnos.

Como mínimo:

Campos obligatorios

No permitir guardar si:

- nombre está vacío;
- especialidad está vacía;
- correo está vacío.

Correo

Comprobar que contenga:

@

Nombre

Comprobar que tenga al menos:

3 caracteres

Podés reutilizar la idea de:

`mostrarMensaje()`

para informar los errores.

14. Mensaje de éxito

Cuando el docente se registre correctamente, mostrar:

`Docente guardado correctamente`

Utilizá el mismo mecanismo que ya implementamos para alumnos.

El mensaje deberá desaparecer automáticamente después de unos segundos.

15. Desafío – Eliminar docentes

Si terminaste la parte anterior, agregá una columna:

`Acciones`

y un botón para eliminar.

Por ejemplo:



El botón deberá almacenar el ID del docente mediante:

`data-id`

Recordá cómo resolvimos el problema de los iconos utilizando:

`closest()`

16. Confirmar antes de eliminar

Antes de eliminar:

```
confirm(...)
```

Deberá aparecer un mensaje similar a:

¿Está seguro de eliminar este docente?

Si el usuario selecciona **Cancelar**, el docente debe permanecer.

Si selecciona **Aceptar**, deberá eliminarse.

17. Actualizar localStorage

Al eliminar un docente, recordá que no alcanza con modificar el array.

Deberás actualizar también:

```
localStorage
```

utilizando:

```
JSON.stringify()
```

Después deberás volver a mostrar la lista.

18. Desafío adicional

Si terminaste todo lo anterior, incorporá la posibilidad de **editar docentes**.

El funcionamiento deberá ser equivalente al módulo Alumnos:


```
Editar
  ↓
Cargar datos en formulario
  ↓
Modificar
  ↓
Actualizar
  ↓
Guardar en localStorage
  ↓
Mostrar nuevamente
```

Utilizá:

```
find()
```

para localizar el docente que se desea modificar.

Antes de comenzar

No copies el módulo de alumnos y simplemente cambies todas las palabras.

Primero identificá:

¿Qué podemos reutilizar conceptualmente?

```
localStorage
JSON.stringify()
JSON.parse()
Date.now()
find()
filter()
trim()
confirm()
closest()
mostrarMensaje()
```

¿Qué debemos cambiar?

alumno



docente

nombre

carrera

correo



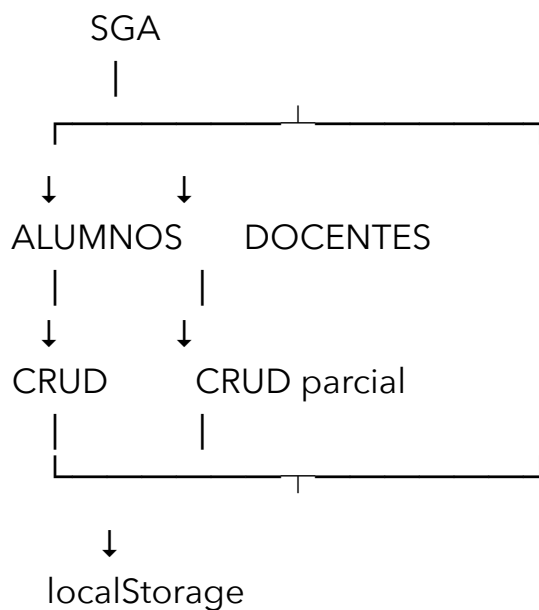
nombre

especialidad

correo

Objetivo final

Si completás la actividad principal, tu SGA debería tener:



Módulo Alumnos

- ☒ Crear
- ☒ Mostrar
- ☒ Editar

- ✓ Eliminar
- ✓ Validar
- ✓ Confirmar eliminación

Módulo Docentes

- ✓ Crear
- ✓ Mostrar
- ✓ Validar
- ✓ Guardar en localStorage
- ★ Eliminar
- ★ Editar